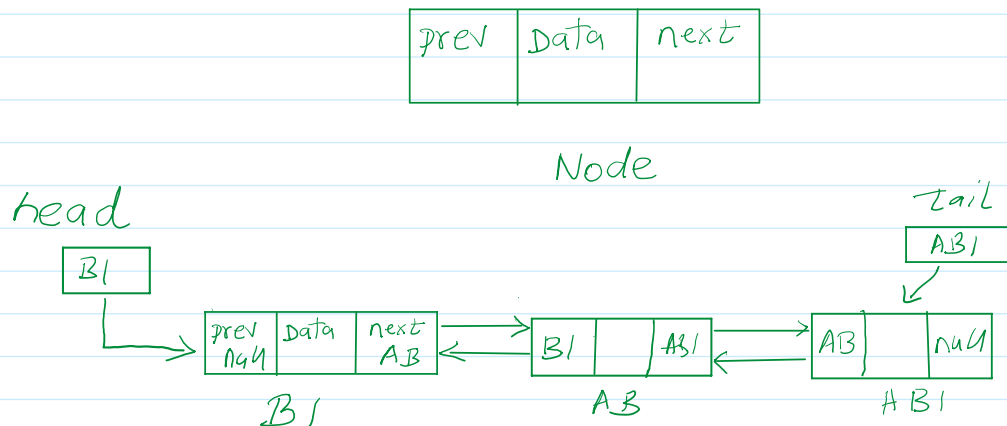


Doubly Linked List

05 September 2026 18:43

A doubly linked list is a linear data structure in which each node contains a references to both the previous and the next node. It supports traversal in both forward and backward direction.



In doubly linked list, the first node is called the head and the previous pointer is null. The last node is called the tail and the next pointer is NULL. Each intermediate node is connected to both its previous and next nodes.

For example, a music playlist application can use a doubly linked list to move to the previous or next song without restarting the playlist.

Characteristics of a doubly linked list

A doubly linked list has the following characteristics:

- Each node contains one data fields and 2 pointers or reference variables.
- It supports traversal in both forward and backward direction.
- The next pointer or the next reference of the last node is null.
- It allows efficient insertion and deletion when the node location is known.

How it works?

Head and Tail: The head links to the first data of the list and the tail links to the last data. These links enhances operations such as adding or removing elements at the head or at the end of the list.

Traversal: It can start from the head and follows the next pointer to move forward or start from the tail and follows the Prev pointer to move backward.

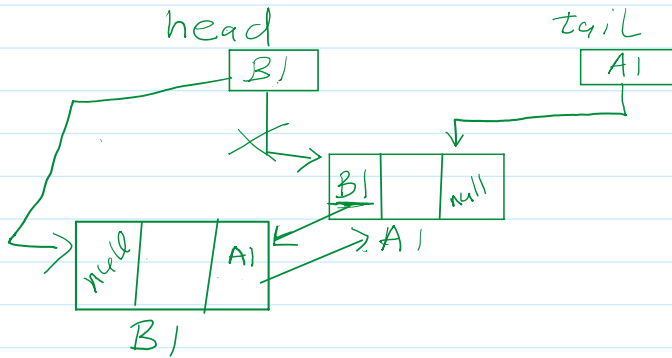
Insertion and deletion: Adding or deleting notes can be done efficiently at any point in the list.

Insertion at head

```
newNode.setNext(head);  
head = newNode;
```

head

Insertion at head



```
newNode.setNext(head);
head = newNode;
public void setNext(DoubleNode node){
    this.next = node;
}
```

```
public void insertNodeAtHead(int data){
    DoubleNode newNode = new DoubleNode(data);
    if(head == null){
        head = tail = newNode;
        return;
    }
    head.setPrev(newNode);
    newNode.setNext(head);
    head = newNode;
}
```

